Ringkasan Metode GEMASTIK NEW-GEN

Model pohon, linear, jarak, neural network, tanpa label, teks, deret waktu, rekomendasi, graf, dan cara menilai model

Cara memakai ringkasan ini

- Sumber: notebook di `_.NEW-GEN/` (199 notebook) dan skor di `KATALOG.md`. Semua gambar dibuat dari **data mainan** dengan `buat_gambar.py`, jadi angkanya nyata tetapi bukan data lomba.
- **Warna direset tiap bab.** Baris kecil di bawah judul bab menjelaskan arti warnanya. Di gambar 2D: **biru** = kelas 0, **jingga** = kelas 1, warna latar = tebakan model (makin pekat = makin yakin).
- Kotak: **Inti** = ide yang membuat hal lain masuk akal; rumus = cukup paham artinya; **Kenapa begitu?** = alasan bertahap; **Diagram** = gambar + cara membaca; **Jebakan lomba** = kesalahan yang sering terjadi; **Di notebook** = baris kode yang benar-benar ada di toolkit; **Kerabatnya** = metode lain di keluarga yang sama.
- **Aturan emas:** model dinilai pada data yang *belum pernah dilihat*. Semua teknik di sini (split, KFold, `max_depth`, `learning_rate`, regularisasi) adalah cara menjamin skor validasi jujur dan model tidak menghafal.

Daftar Isi

1 Peta Besar: Dari Soal ke Metode **2**

2 Model Berbasis Pohon **3**

2.1 Decision Tree . . . 3

2.2 Random Forest dan ExtraTrees . . . 3

2.3 Gradient Boosting: XGBoost, LightGBM, CatBoost . . . 4

3 Model Linear dan Model Jarak **5**

3.1 Regresi logistik (dan Ridge, LinearSVC) . . . 5

3.2 K-Nearest Neighbors (KNN) . . . 6

4 Neural Network dan Transfer Learning **7**

5 Tanpa Label: Clustering, Reduksi Dimensi, Anomali **7**

5.1 KMeans . . . 7

5.2 PCA, UMAP, t-SNE . . . 8

5.3 Isolation Forest . . . 9

6 Teks: Dari Kata Menjadi Angka **10**

7 Deret Waktu: Meramal dari Masa Lalu **10**

7.1 Exponential smoothing . . . 10

7.2 SARIMAX . . . 11

7.3 Fitur lag + model tabel . . . 11

8 Rekomendasi dan Graf **12**

8.1 Matrix factorization (SVD) . . . 12

8.2 Deteksi komunitas (Louvain) . . . 12

9 Menilai dan Memeras Skor **13**

9.1 Validasi silang (KFold) . . . 13

9.2 Precision, recall, F1, dan threshold . . . 13

9.3 Ensembling, tuning, dan SHAP . . . 14

Lembar ringkas satu halaman **15**

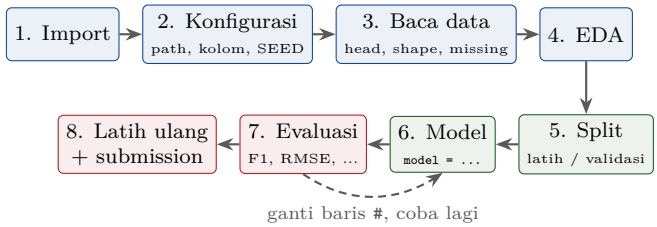
Kesalahan pemula yang perlu diingat **15**

1 Peta Besar: Dari Soal ke Metode

Warna bab ini: menyiapkan data model menilai

Setiap notebook NEW-GEN punya bentuk yang sama: 8 sel berurutan. Yang berubah antar-soal hanya isi sel Model.

Diagram 1: Alur satu notebook



Sel 1–4 menyiapkan data, sel 5–6 melatih model, sel 7–8 menilai dan mengirim. **Cara membaca:** panah putus-putus = putaran percobaan; ganti model, jalankan ulang, bandingkan skor.

Inti. Satu notebook = satu metode. Di sel Model ada **satu baris** `model = ...` yang aktif, dan alternatifnya ditulis di baris `#`. Mencoba model lain = pindahkan tanda `#`, lalu *Run All*. Skor tiap alternatif pada data demo ada di `KATALOG.md`.

Soal jenis apa, coba metode apa

Jenis soal	Contoh	Model aktif → alternatif kuat	Notebook
Tabel → kelas	pelanggan berhenti?	LogisticRegression → LightGBM, XGBoost, CatBoost	`tabular/klasifikasi`
Tabel → angka	harga rumah	LightGBM → XGBoost, Ridge	`tabular/regresi`
Teks → kelas	sentimen ulasan	TF-IDF + LinearSVC → embedding, fine-tune IndoBERT	`text/klasifikasi_tfidf`
Gambar → kelas	citra medis	CNN pretrained (ResNet)	`medis/klasifikasi_citra`
Cari objek di gambar	hitung kendaraan	YOLO	`vision/deteksi`
Ramal masa depan	penjualan bulan depan	fitur lag + RandomForest → LightGBM; SARIMAX	`timeseries/peramalan_ml`
Kelompok tanpa label	segmen pelanggan	KMeans (+ PCA untuk gambar)	`unsupervised/clustering`
Cari yang aneh	transaksi curang	IsolationForest → LOF, ECOD	`timeseries/anomali`
Rekomendasi	tebak rating film	SVD (matrix factorization)	`recsys/prediksi_rating`
Jaringan / graf	komunitas	Louvain	`graph/komunitas`

Empat konsep dasar

Fitur dan label	X = kolom yang kita tahu (umur, gaji, jumlah keluhan); y = yang ingin ditebak (berhenti atau tidak). Model mencari hubungan $X \rightarrow y$.
Klasifikasi vs regresi	klasifikasi menebak **kelas**; regresi menebak **angka**. Banyak model punya dua versi: `XGBClassifier` dan `XGBRegressor`.
Latih vs validasi	sebagian data disembunyikan dari model; skor dihitung di sana karena di lomba model juga dinilai pada data yang belum pernah dilihat.
Overfitting	model terlalu rumit → menghafal data latih termasuk noise-nya: skor latih tinggi, skor validasi turun (Diagram 3).

Jebakan lomba. Kebocoran data (leakage). Skor validasi tinggi sekali tetapi skor lomba jeblok, biasanya karena informasi dari data validasi/test bocor ke latihan: `StandardScaler` di-`fit` pada seluruh data, kolom yang sebenarnya hasil dari target (misal "tanggal_berhenti" untuk menebak berhenti), atau deret waktu yang diacak. Karena itu toolkit memakai `Pipeline`: preprocessing ikut di-`fit` hanya pada data latih.

2 Model Berbasis Pohon

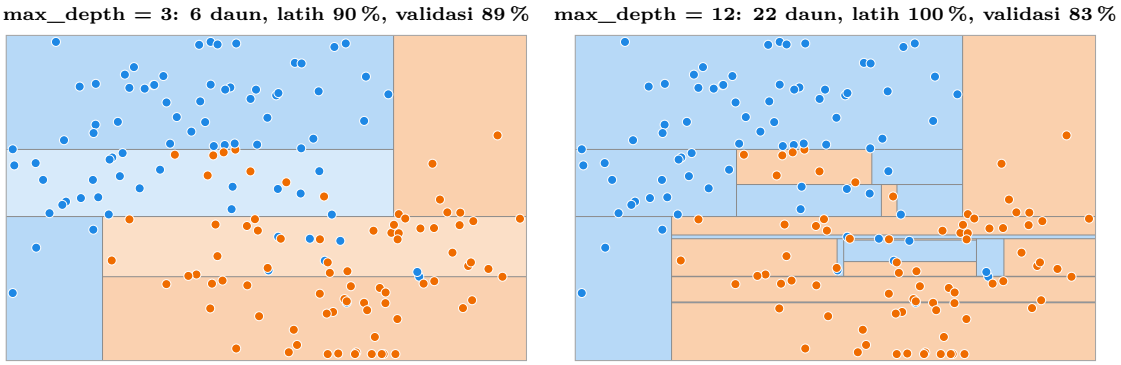
Warna bab ini: kelas 0 kelas 1 tebakan model / validasi sisa kesalahan / latih

Keluarga paling sering menang di data tabel. Semuanya dibangun dari satu bahan: pohon keputusan.

2.1 Decision Tree

Inti. Model bertanya ya/tidak berulang kali, misalnya "umur < 30?" lalu "gaji < 5 juta?", sampai isi tiap kotak cukup seragam. Setiap pertanyaan memotong ruang data dengan **garis lurus tegak atau mendatar**. Tebakan = kelas mayoritas di kotak tempat titik jatuh.

Diagram 2: Pohon dangkal vs pohon dalam

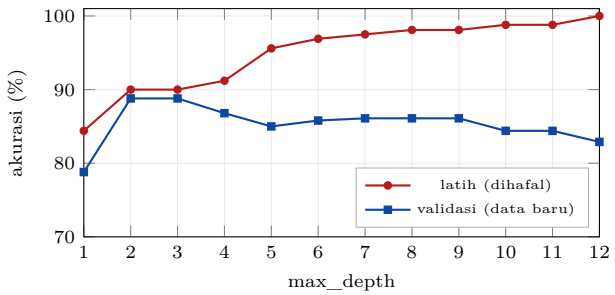


Garis tipis = potongan pohon. **Cara membaca:** kiri, paling banyak 3 pertanyaan berurutan, kotaknya besar dan agak kasar. Kanan, sampai 12 pertanyaan berurutan, banyak kotak kecil yang dibuat hanya untuk satu-dua titik (yang sebenarnya noise). Akurasi latih naik ke 100 %, tetapi akurasi validasi justru turun.

Gini (ukuran "campur aduk") $\text{Gini} = 1 - \sum_k p_k^2$ (0 = murni satu kelas, 0,5 = campur rata 2 kelas)

Kenapa begitu? (cara memilih potongan) (1) Di tiap kotak, model mencoba *semua* garis yang mungkin pada *semua* fitur. (2) Untuk tiap garis, hitung Gini kiri dan kanan (dibobot jumlah titik). (3) Pilih garis yang Gini-nya paling kecil. (4) Ulangi di tiap kotak baru sampai `max_depth` tercapai atau kotak sudah murni. Tidak ada rumus "garis terbaik"; pohon mencoba satu per satu.

Diagram 3: Overfitting terlihat di kurva

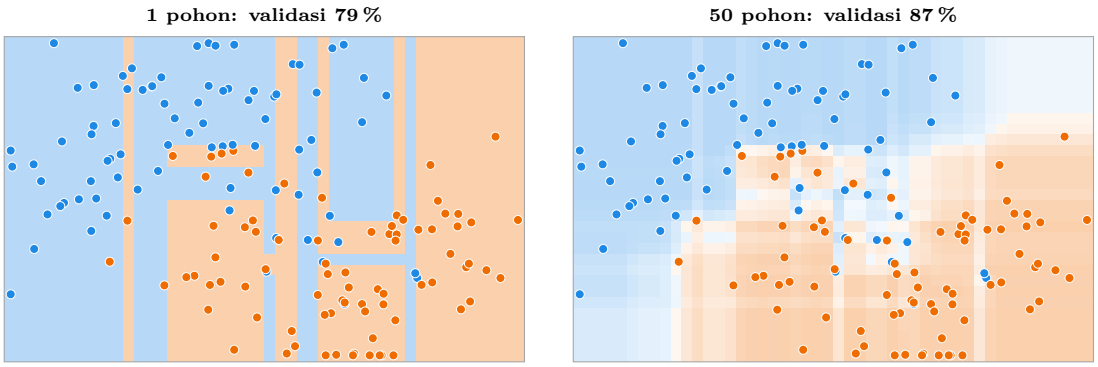


Cara membaca: garis merah (latih) terus naik mendekati 100 % karena pohon makin pandai menghafal. Garis biru (validasi) berhenti naik lalu turun. Kedalaman terbaik = puncak garis biru, bukan garis merah.

2.2 Random Forest dan ExtraTrees

Inti. Buat ratusan pohon yang dalam. Tiap pohon hanya melihat **sampel acak** dari baris data (bootstrap) dan **fitur acak** di tiap potongan, jadi kesalahannya berbeda-beda. Tebakan akhir = **rata-rata suara** semua pohon.

Diagram 4: Satu pohon vs hutan 50 pohon



Cara membaca: satu pohon dalam (kiri) membuat batas kotak-kotak tajam dan percaya diri walaupun salah. Rata-rata 50 pohon (kanan) membuat batas halus; warna pudar di perbatasan = pohon-pohon tidak sepakat. Validasi naik dari 79,4 % ke 87,4 %.

Kenapa begitu? (kenapa rata-rata pohon lebih baik) Satu pohon dalam punya *variansi* tinggi: ganti sedikit datanya, bentuknya berubah jauh. Kalau kesalahan tiap pohon acak dan tidak searah, merata-ratakan banyak pohon membuat kesalahan saling meniadakan, sedangkan pola yang benar (yang dilihat hampir semua pohon) tetap tinggal. Mirip bertanya ke 500 orang yang masing-masing membaca sebagian berita.

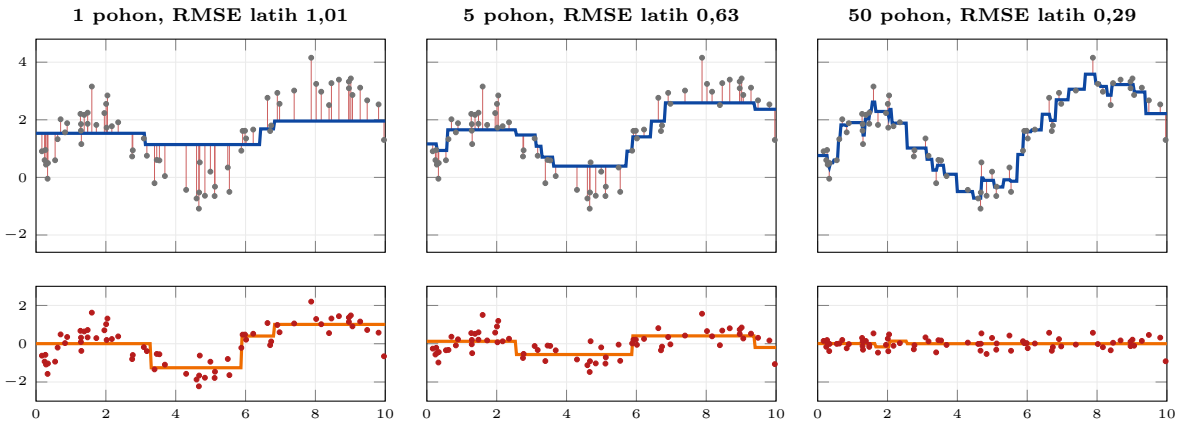
Kerabatnya. `ExtraTrees` = Random Forest yang titik potongnya juga diacak (tidak dicari yang terbaik), jadi lebih cepat dan kadang lebih tahan noise. Kenop: `n_estimators` (300–500 cukup; lebih banyak tidak merusak, hanya lebih lambat), `max_features`.

2.3 Gradient Boosting: XGBoost, LightGBM, CatBoost

Inti. Pohon dibuat **satu per satu**. Pohon pertama menebak kasar. Pohon kedua *tidak* menebak target, tetapi menebak **sisa kesalahan** (residual) pohon pertama. Pohon ketiga menebak sisa kesalahan berikutnya, dan seterusnya. Tebakan akhir = jumlah semua pohon. Setiap pohon dikali `learning_rate` supaya langkahnya kecil dan hati-hati.

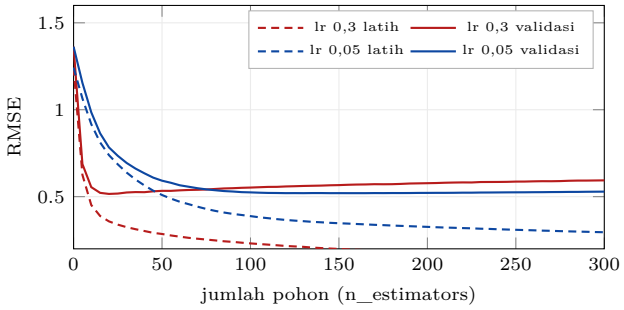
boosting $$F_m(x) = F_{m-1}(x) + \eta\, h_m(x), \qquad h_m \text{ dilatih menebak } y - F_{m-1}(x)$$

Diagram 5: Boosting ronde demi ronde (regresi 1 fitur)



Baris atas: titik abu = data; garis biru = jumlah semua pohon sejauh ini; garis tegak merah = sisa kesalahan tiap titik. **Baris bawah:** sisa kesalahan itu digambar sendiri (titik merah), dan garis tangga jingga = pohon kecil berikutnya yang dilatih menebaknya. Dari 1 ke 5 ke 50 pohon, garis biru makin menempel ke pola data dan sisa kesalahan makin acak (tidak berpola lagi).

Diagram 6: Learning rate besar vs kecil



Cara membaca: putus-putus = latih, garis penuh = validasi. `learning_rate` 0,3 (merah) turun cepat lalu validasinya mulai naik lagi (overfitting). 0,05 (biru) turun pelan, butuh lebih banyak pohon, tetapi titik terendah validasinya setara atau lebih baik dan lebih stabil.

Kenapa begitu? (kenapa menebak sisa kesalahan) (1) Pohon dangkal sendirian lemah, hanya bisa membuat 4–8 anak tangga. (2) Kalau pohon berikutnya fokus ke bagian yang masih salah, kemampuan bertambah tepat di tempat yang dibutuhkan. (3) Menjumlahkan ratusan tangga kecil menghasilkan bentuk apa pun. (4) `learning_rate` kecil = tiap pohon hanya memperbaiki sebagian, sehingga satu pohon yang keliru tidak merusak semuanya. Nama "gradient" karena sisa kesalahan $y - F$ adalah arah turun (gradien) dari error kuadrat.

	XGBoost	**LightGBM**	**CatBoost**	**HistGradient-Boosting**
Cara tumbuh	lapis demi lapis, rata	daun demi daun: yang paling untung dipecah dulu	pohon simetris	mirip LightGBM
Kelebihan	stabil, paling populer	sangat cepat di data besar	kolom kategori tanpa encoding	bawaan sklearn
Kenop	`n_estimators`, `learning_rate`, `max_depth`	`n_estimators`, `learning_rate`, `num_leaves`	`iterations`, `learning_rate`, `depth`	`max_iter`, `learning_rate`

Di notebook: tabular/regresi (baris model aktif)

```
model = LGBMRegressor(n_estimators=1000, learning_rate=0.03, random_state=SEED, verbose=-1)
# model = XGBRegressor(n_estimators=500, learning_rate=0.03, random_state=SEED)
# model = CatBoostRegressor(iterations=500, random_seed=SEED, verbose=0)
```

Aturan praktis: `learning_rate` 0,03–0,1; `n_estimators` 300–1000; pohon dangkal (depth 3–8 atau `num_leaves` 15–63). Turunkan `learning_rate` ⇒ naikkan `n_estimators`.

Jebakan lomba. Random Forest dan boosting *terdengar* mirip karena sama-sama "banyak pohon". Bedanya: Random Forest = pohon **bersamaan, independen, dirata-rata** (menambah pohon tidak bikin overfitting). Boosting = pohon **berurutan, saling memperbaiki, dijumlah** (terlalu banyak pohon *bisa* overfitting, lihat Diagram 6).

3 Model Linear dan Model Jarak

Warna bab ini: kelas 0 kelas 1 bobot dan skor

Cepat, mudah dijelaskan, dan baseline yang kuat. Andalan untuk teks dan embedding karena fiturnya sangat banyak.

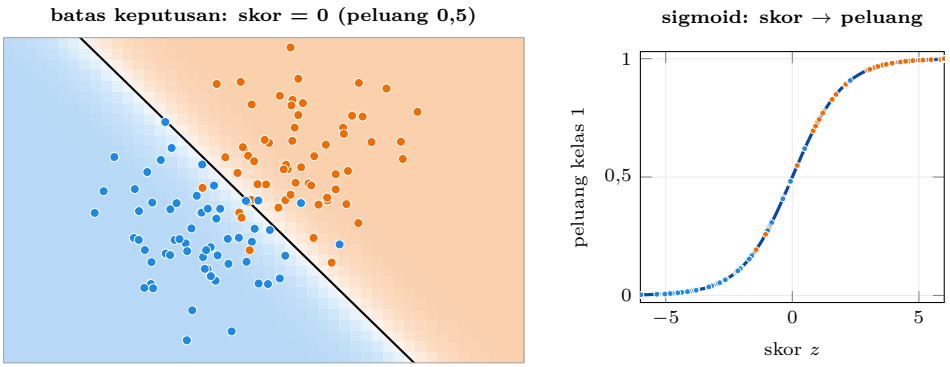
3.1 Regresi logistik (dan Ridge, LinearSVC)

Inti. Tiap fitur diberi **bobot**, lalu dijumlah jadi satu **skor**. Untuk klasifikasi, skor diubah jadi peluang 0–1 lewat kurva S (sigmoid). Batas keputusan = tempat skor = 0 (peluang 0,5), dan bentuknya **selalu garis lurus** (bidang datar kalau fiturnya banyak).

skor $$z = b_1 x_1 + b_2 x_2 + \cdots + c$$

sigmoid $$p = \frac{1}{1 + e^{-z}}$$

Diagram 7: Garis batas dan kurva S



Cara membaca: kiri, garis hitam = semua titik yang skornya 0. Hasil latihan di sini: $b_1 = 3{,}65$, $b_2 = 3{,}74$, $c = 0{,}74$, akurasi 93,3 %. Kanan, tiap titik diletakkan menurut skornya; makin jauh dari 0, makin yakin. Titik biru yang jatuh di skor positif (atau jingga di skor negatif) adalah yang salah tebak.

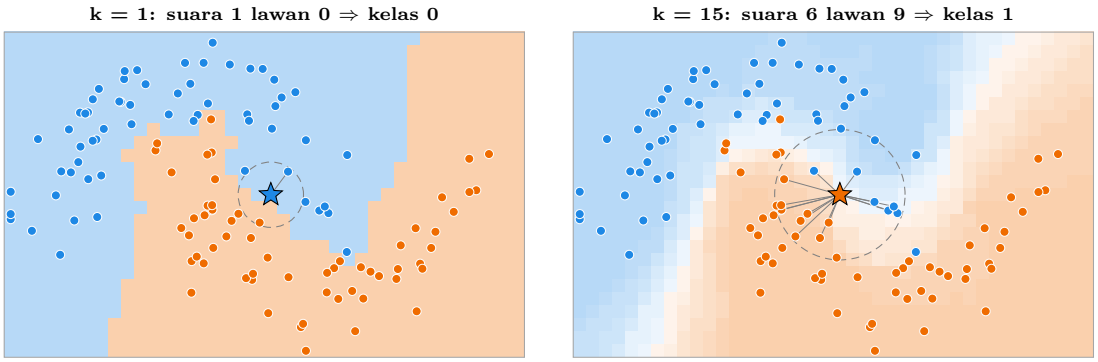
Kenapa begitu? (bagaimana bobot dicari) (1) Mulai dari bobot sembarang. (2) Hitung kesalahan (*log-loss*: hukuman besar kalau yakin tapi salah). (3) Geser bobot sedikit ke arah yang mengurangi kesalahan (*gradient descent*). (4) Ulangi ratusan kali. Sklearn melakukannya otomatis saat `fit`; `max_iter=2000` = batas jumlah langkah.

Kerabatnya. `Ridge` dan `Lasso` = versi regresi (tebak angka); Lasso bisa membuat bobot fitur tak berguna jadi tepat 0. `LinearSVC` dan `SGDClassifier` = garis lurus juga, cara melatih berbeda. `SVC` dengan kernel bisa melengkung karena ruang datanya "ditekuk" dulu. Kenop bersama: `C` (atau `alpha` di Ridge). `C` kecil = bobot ditahan, lebih aman dari overfitting.

3.2 K-Nearest Neighbors (KNN)

Inti. Tidak ada pelatihan. Untuk menebak titik baru, cari k **tetangga terdekat** di data latih, lalu ambil suara terbanyak (atau rata-rata untuk regresi).

Diagram 8: Pengaruh k



Bintang = titik yang ditanya; garis = tetangga yang ikut memilih; lingkaran putus-putus = batas tetangga terjauh. **Cara membaca:** $k = 1$ (kiri) membuat batas berliku-liku dan pulau-pulau kecil karena satu titik aneh langsung berpengaruh. $k = 15$ (kanan) membuat batas halus. Tebakan untuk bintang yang sama pun bisa berubah.

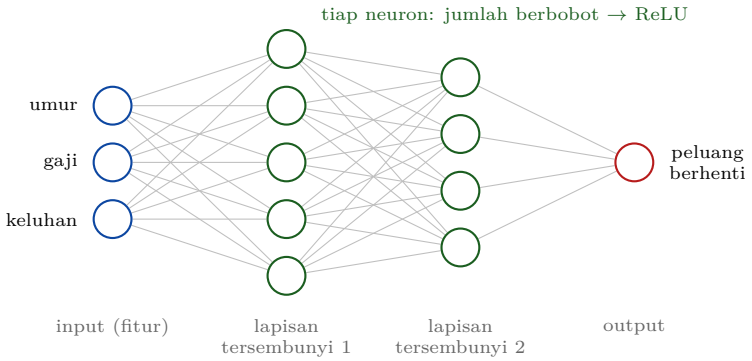
Jebakan lomba. Model berbasis jarak (KNN, SVC, KMeans, juga regresi logistik saat dilatih) **wajib** diberi `StandardScaler()` dulu. Tanpa itu, kolom "gaji" (jutaan) menenggelamkan kolom "umur" (puluhan) karena jarak-nya dihitung dari angka mentah. Model pohon tidak butuh scaler.

4 Neural Network dan Transfer Learning

Warna bab ini: input lapisan tersembunyi output

Dipakai untuk teks, gambar, audio, dan deret waktu. Di toolkit hampir selalu memakai model yang sudah dilatih orang lain.

Diagram 9: MLP: regresi logistik yang disusun berlapis



Setiap lingkaran = satu neuron = jumlah berbobot dari lapisan sebelumnya, lalu ReLU (angka negatif dijadikan 0). Setiap garis = satu bobot. **Cara membaca:** lapisan awal mendeteksi pola sederhana, lapisan berikutnya menggabungkannya jadi pola rumit. Saat `fit`, semua bobot diperbaiki bersama (*backpropagation*), mirip gradient descent di bab regresi logistik.

Jenis	Idenya	Notebook
MLP	seperti diagram di atas; untuk tabel atau embedding. `hidden_layer_sizes=(256,)` = satu lapisan 256 neuron	`tabular/deep_klasifikasi`
CNN (ResNet)	filter kecil 3×3 digeser ke seluruh gambar mencari garis → sudut → tekstur → bentuk	`medis/klasifikasi_citra`
YOLO	CNN yang langsung menebak kotak + kelas objek dalam satu kali lihat	`vision/deteksi`
LSTM	membaca urutan satu per satu sambil membawa "ingatan"	`timeseries/deep_lstm`
Transformer	tiap kata "melihat" semua kata lain sekaligus (*attention*); IndoBERT, Chronos	`text/klasifikasi_finetune`

Inti. Transfer learning. Melatih neural network dari nol butuh jutaan data dan GPU berhari-hari. Toolkit memakai model *pretrained*, lalu salah satu dari dua cara: (1) **ambil embedding** (ringkasan berupa ratusan angka) dan latih LogisticRegression di atasnya: cepat dan murah; (2) **fine-tune**: lanjutkan latihan sebentar dengan data lomba: lebih akurat, butuh GPU.

Di notebook: text/klasifikasi_embedding (cara 1)

```
encoder = SentenceTransformer(EMBEDDING_MODEL)      # model pretrained, tidak dilatih ulang
X_train = encoder.encode(train_texts)               # tiap kalimat -> ratusan angka
model = LogisticRegression(C=1.0, max_iter=2000)    # model biasa di atas embedding
```

Kenapa begitu? (kenapa embedding berguna) Model pretrained sudah membaca jutaan kalimat atau gambar, sehingga ia "tahu" bahwa *murah* dan *terjangkau* mirip, atau bahwa telinga runcing + kumis = kucing. Embedding menyimpan pengetahuan itu sebagai posisi: yang mirip berdekatan. Model sederhana di atasnya tinggal menarik garis.

5 Tanpa Label: Clustering, Reduksi Dimensi, Anomali

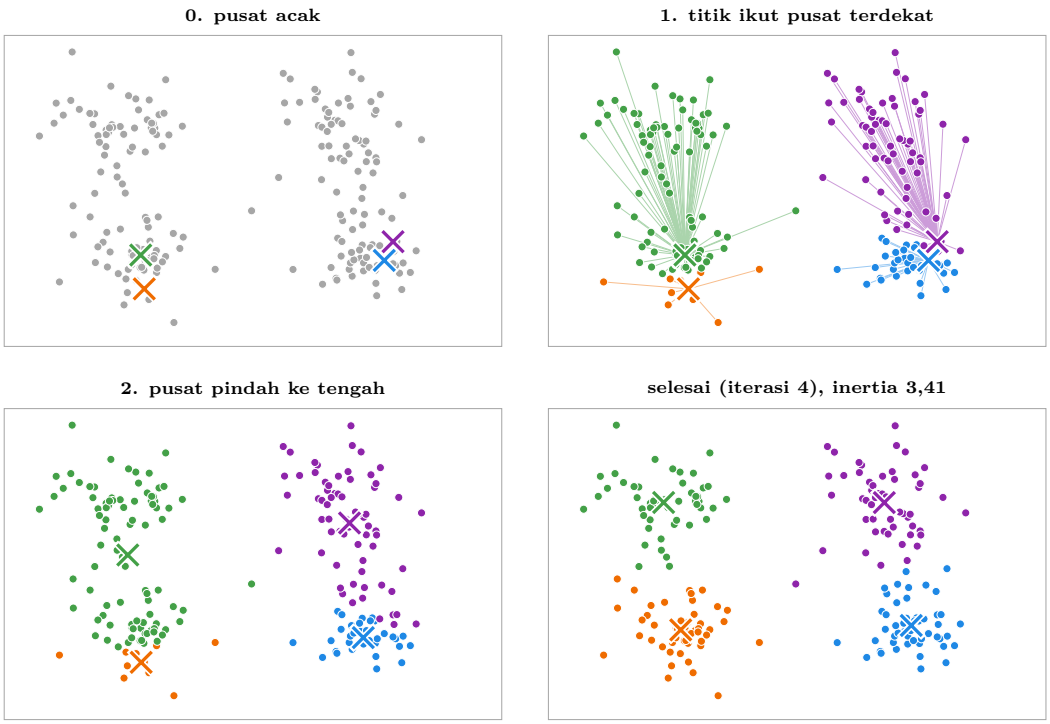
Warna bab ini: data hasil proyeksi / titik terpilih potongan acak

Tidak ada kolom y. Model mencari struktur sendiri; kita yang menafsirkan hasilnya.

5.1 KMeans

Inti. Taruh k pusat secara acak, lalu ulangi dua langkah: **(1)** tiap titik ikut pusat terdekat, **(2)** tiap pusat pindah ke tengah (rata-rata) anggotanya. Berhenti saat tidak ada titik yang pindah kelompok.

Diagram 10: KMeans langkah demi langkah ($k = 4$)



× = pusat klaster. **Cara membaca:** panel 0, pusat diambil dari titik acak (dua pusat bisa jatuh di kelompok yang sama). Panel 1, garis tipis menunjukkan tiap titik memilih pusat terdekat. Panel 2, pusat bergeser ke tengah anggotanya. Setelah 4 putaran, hasilnya stabil.

inertia $\quad \text{inertia} = \sum_{\text{titik}} \text{jarak(titik, pusatnya)}^2 \quad \text{(makin kecil = makin rapat)}$

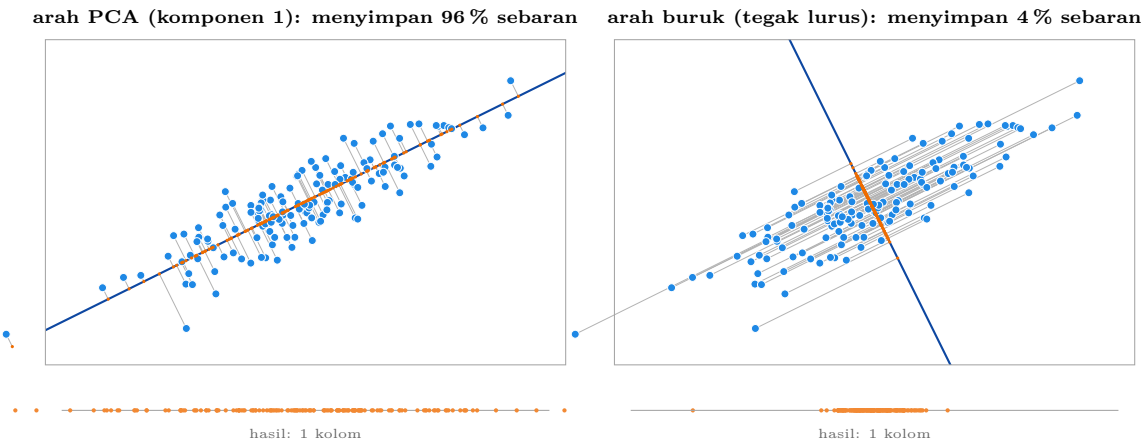
Jebakan lomba. KMeans **selalu** memberi jawaban, bahkan untuk k yang salah. Memilih k adalah tugas kita: grafik *elbow* (inertia vs k, cari "siku") atau *silhouette score* (makin tinggi makin baik). Skalakan fitur dulu. Nama klaster ("pembeli hemat") kita yang memberi, dengan melihat rata-rata fitur tiap klaster.

Kerabatnya. `DBSCAN`/`HDBSCAN`: klaster = daerah padat, jumlahnya otomatis, titik sepi diberi label noise (-1); cocok untuk bentuk tak bulat (GPS, trajektori). `AgglomerativeClustering`: gabung dua kelompok terdekat berulang-ulang (dendrogram). `GaussianMixture`: seperti KMeans tetapi tiap titik punya peluang ikut tiap klaster.

5.2 PCA, UMAP, t-SNE

Inti. Meringkas banyak kolom jadi sedikit kolom baru. PCA mencari **arah tempat data paling menyebar**, lalu memproyeksikan (menjatuhkan) setiap titik ke arah itu. Komponen kedua = arah tersebar berikutnya yang tegak lurus dengan yang pertama.

Diagram 11: Arah yang baik vs arah yang buruk



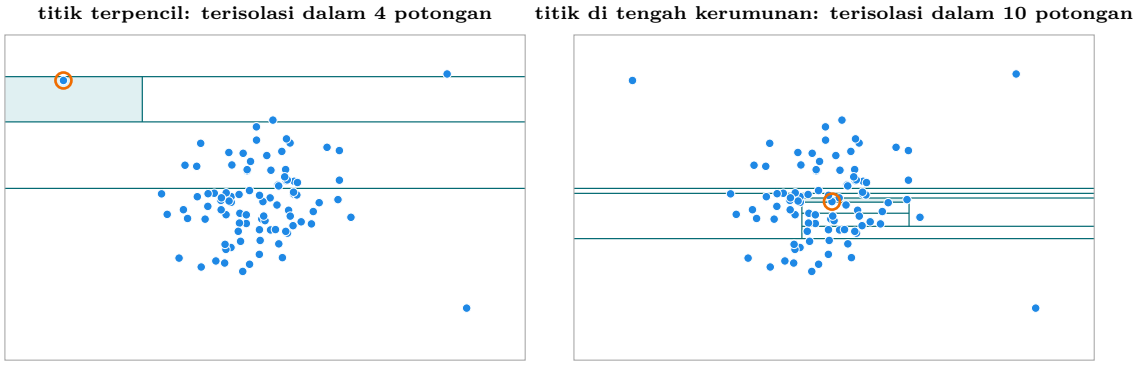
Garis tipis = jalur jatuhnya titik ke garis biru; titik jingga = posisi baru. Strip di bawah = data setelah tinggal 1 kolom. **Cara membaca:** di arah PCA (kiri) titik-titik tetap terpisah lebar, 96 % sebaran tersimpan. Di arah tegak lurus (kanan) semuanya menumpuk, hanya 4 % tersimpan. Di sklearn angka ini ada di `explained_variance_ratio_`.

Kerabatnya. **UMAP** dan **t-SNE** hanya untuk *melihat*. Keduanya menjaga agar tetangga dekat tetap dekat, jadi kelompok tampak jelas di gambar 2D, tetapi jarak antar-kelompok dan ukuran kelompok di gambar tidak bermakna. PCA juga dipakai untuk mengecilkan embedding (misal 768 → 50 kolom) sebelum clustering.

5.3 Isolation Forest

Inti. Potong data secara acak berulang-ulang. Titik aneh yang jauh dari kerumunan **cepat terpisah sendirian**; titik normal di tengah kerumunan butuh banyak potongan. **Sedikit potongan = anomali.**

Diagram 12: Berapa potongan untuk mengisolasi satu titik



Cincin jingga = titik yang sedang diisolasi; garis = potongan acak; kotak berarsir = kotak terakhir yang hanya berisi titik itu. **Cara membaca:** rata-rata 300 kali percobaan, titik terpencil butuh 2,9 potongan, titik di tengah butuh 11,5 potongan. IsolationForest mengulang ini dengan ratusan pohon acak dan mengubah rata-rata panjang jalur jadi skor anomali.

Kerabatnya. `LocalOutlierFactor`: titik yang daerahnya jauh lebih sepi daripada daerah tetangganya. `ECOD`, `HBOS` (pyod): seberapa ekstrem nilai tiap kolom. Kenop `contamination` = perkiraan persen data yang anomali; ini tebakan kita, jadi selalu cek baris yang ditandai secara manual (`tabular/cek_outlier_input` menyimpannya ke `baris_curiga.csv`).

6 Teks: Dari Kata Menjadi Angka

Warna bab ini: nilai TF-IDF

Model hanya mengerti angka. Langkah pertama setiap notebook teks adalah mengubah kalimat menjadi tabel angka.

Inti. **TF-IDF** memberi nilai tinggi pada kata yang **sering di dokumen ini** (TF) tetapi **jarang di dokumen lain** (IDF). Kata yang muncul di mana-mana ("dan", "barang") bernilai kecil karena tidak membedakan apa-apa. Setiap kata menjadi satu kolom; LogisticRegression/LinearSVC lalu memberi bobot ke tiap kolom.

seperti sklearn, smooth_idf

$$\text{TF} = \frac{\text{jumlah kata di dokumen}}{\text{panjang dokumen}}, \qquad \text{IDF} = \ln \frac{1+N}{1+\text{df}} + 1, \qquad \text{TF-IDF} = \text{TF} \times \text{IDF}$$

</> Di notebook: 4 ulasan, nilai kata di dokumen 4 "barang bagus harga murah murah"

Dokumen: (1) pengiriman cepat dan barang bagus; (2) barang rusak dan pengiriman lambat sekali; (3) pelayanan ramah dan barang bagus sekali; (4) barang bagus harga murah murah. $N = 4$; df = berapa dokumen memuat kata itu.

kata	TF	df	IDF	TF-IDF
murah	$2/5 = 0{,}40$	1 dari 4	1,92	0,767
harga	$1/5 = 0{,}20$	1 dari 4	1,92	0,383
bagus	$1/5 = 0{,}20$	3 dari 4	1,22	0,245
barang	$1/5 = 0{,}20$	4 dari 4	1,00	0,200

"murah" paling tinggi: muncul dua kali dan hanya di dokumen ini. "barang" paling rendah: ada di semua dokumen.

Kerabatnya. `ngram_range=(1,2)`: pasangan kata ikut dihitung, sehingga "tidak bagus" jadi satu kolom. `min_df=2`: buang kata yang muncul di kurang dari 2 dokumen. `analyzer="char_wb"`: potongan huruf, tahan salah ketik dan bahasa gaul. `sublinear_tf=True`: kata yang muncul 10 kali tidak dianggap 10 kali lebih penting.

Embedding dan cosine similarity

TF-IDF tidak tahu bahwa "murah" dan "terjangkau" mirip. Embedding (SentenceTransformer) tahu, karena model pretrained-nya sudah membaca jutaan kalimat. Kemiripan dua kalimat diukur dari sudut antara dua vektornya:

cosine similarity

$$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\,\|\vec{b}\|} \qquad (1 = \text{searah / sangat mirip},\ 0 = \text{tidak berhubungan})$$

Dipakai di `text/retrieval` (cari dokumen paling mirip pertanyaan), `text/rag_llm`, dan `preprocessing/peta_semantik`.

Jebakan lomba. Jangan `fit` `TfidfVectorizer` pada gabungan latih + test lalu melaporkan skor validasi: IDF ikut "melihat" data validasi. Masukkan vectorizer ke `Pipeline` atau `fit` pada data latih saja, lalu `transform` yang lain.

7 Deret Waktu: Meramal dari Masa Lalu

Warna bab ini: α kecil α besar kejadian khusus

Masa depan ditebak dari masa lalu. Tiga cara di toolkit: smoothing/SARIMAX, fitur lag + model tabel, dan model besar (Chronos).

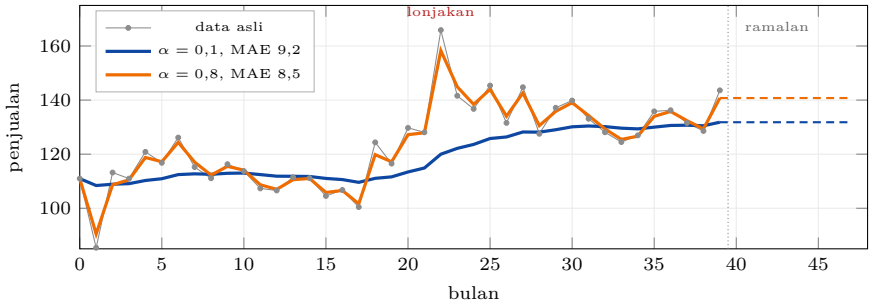
7.1 Exponential smoothing

Inti. Rata-rata yang lebih berat ke data terbaru. **Level** = perkiraan "nilai normal saat ini"; ramalan ke depan = level terakhir. α mengatur seberapa cepat level mengikuti data baru.

level

$$\ell_t = \alpha\, y_t + (1-\alpha)\,\ell_{t-1}, \qquad \hat{y}_{t+1} = \ell_t$$

Diagram 13: Pengaruh α



Cara membaca: $\alpha = 0{,}1$ (biru) tenang tetapi selalu tertinggal di belakang data yang terus naik, MAE 9,2. $\alpha = 0{,}8$ (jingga) cepat mengikuti, tetapi ikut melonjak di bulan 22, MAE 8,5. Ramalan keduanya datar karena model ini tidak mengenal tren.

Kenapa begitu? (kenapa ada Holt dan Holt-Winters) Simple smoothing hanya punya level, jadi ramalannya datar. **Holt** menambah komponen *tren* (naik berapa per bulan), **Holt-Winters** menambah komponen *musim* (pola yang berulang tiap 12 bulan atau tiap 7 hari). Di statsmodels: `ExponentialSmoothing(trend="add", seasonal="add", seasonal_periods=12)`.

7.2 SARIMAX

AR	*autoregressive*: nilai sekarang mirip nilai beberapa periode lalu
I	*integrated*: modelkan selisih ($y_t - y_{t-1}$), bukan nilai mentah, supaya tren hilang
MA	*moving average*: koreksi dari kesalahan ramalan periode lalu
S	*seasonal*: AR/I/MA yang sama tetapi untuk jarak satu musim (12 bulan lalu)
X	*exogenous*: faktor luar yang diketahui (libur, promo, harga)

7.3 Fitur lag + model tabel

Inti. Cara yang sering paling kuat di lomba: ubah deret jadi **tabel biasa**. Tiap baris berisi nilai bulan lalu (`lag_1`), tahun lalu (`lag_12`), rata-rata beberapa bulan sebelumnya, bulan ke berapa, lalu latih RandomForest/LightGBM biasa.

bulan	jual (y)	`lag_1`	`lag_12`	`rata_3`
2024-01	120	–	–	–
2024-02	132	120	–	–
2024-03	128	132	–	–
2024-04	140	128	–	126,7
…	…	…	…	…
2025-01	151	145	120	143,0

Jebakan lomba. (1) Deret waktu **tidak boleh diacak** saat split; validasi harus sesudah data latih (`TimeSeriesSplit`, Diagram 14). (2) `rata_3` harus dihitung dari bulan *sebelumnya* (`shift(1)` dulu, baru `rolling(3)`); kalau bulan ini ikut dihitung, model mengintip jawaban. (3) Saat meramal beberapa langkah, `lag_1` bulan depan belum diketahui; pakai hasil ramalan sebelumnya atau model terpisah per horizon.

8 Rekomendasi dan Graf

Warna bab ini: tebakan model komunitas 1 komunitas 2 komunitas 3

Dua jenis data yang bentuknya bukan tabel biasa: matriks orang × barang, dan jaringan simpul + sisi.

8.1 Matrix factorization (SVD)

Inti. Tiap orang dan tiap film diringkas jadi beberapa angka "selera" (faktor). Rating tebakan = rata-rata umum + kebiasaan orang (pelit/murah memberi nilai) + kebiasaan film (memang disukai/tidak) + kecocokan selera orang dan film.

seperti `surprise.SVD` $\hat{r}_{u,i} = \mu + b_u + b_i + \vec{p}_u \cdot \vec{q}_i$

Di notebook: 5 orang × 6 film, 2 faktor (dilatih dengan SGD di `buat_gambar.py`)

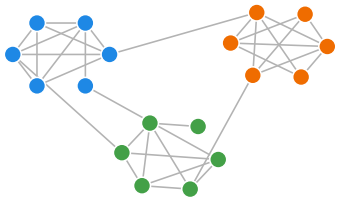
	Horor A	Horor B	Komedi A	Komedi B	Drama A	Drama B
Ani	5	4	1	***1,4***	2	***1,6***
Budi	4	***3,9***	2	1	***2,4***	2
Citra	1	2	5	4	***4,3***	4
Dodi	***1,4***	1	4	5	4	***3,5***
Eka	2	***3,2***	***5,0***	4	5	5

Angka tegak = rating asli; angka ***miring biru*** = tebakan untuk sel kosong. Contoh Ani × Komedi B: $\mu + b_u + b_i + \vec{p} \cdot \vec{q} = 3{,}19 + (-0{,}28) + (-0{,}15) + (-1{,}36) = 1{,}39$. Ani suka horor dan memberi Komedi A nilai 1, jadi kecocokan seleranya negatif.

Kerabatnya. `KNNBaseline` (orang/film yang mirip), `SVDpp`, `NMF`. Untuk data klik tanpa rating: `ALS` dan `BPR` (`recsys/top_n`). Baseline **popularitas** (rekomendasikan yang paling banyak dibeli) sering sulit dikalahkan; di data demo toolkit ia justru menang (`KATALOG.md`).

8.2 Deteksi komunitas (Louvain)

Inti. Cari kelompok simpul yang **banyak saling terhubung di dalam** dan **sedikit ke luar**. Louvain memindah simpul antar kelompok selama skor *modularitas* naik, lalu menggabungkan tiap kelompok jadi satu simpul besar dan mengulang.



modularitas $Q = \sum_c \left[\frac{L_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right]$

L_c = sisi di dalam kelompok c; d_c = jumlah derajat simpul di c; m = total sisi (35). Pembagian di gambar: $Q = 0{,}551$. Semua dijadikan satu kelompok: $Q = 0{,}000$. Makin tinggi (maks. 1) = kelompok makin jelas.

Kerabatnya. Fitur graf untuk model tabel: **degree** (jumlah teman), **PageRank** (dihubungi simpul penting), **betweenness** (sering jadi jembatan). Fitur ini lalu masuk LightGBM biasa (`graph/klasifikasi_node`). **GCN**: neural network yang membuat tiap simpul mengumpulkan informasi dari tetangganya, beberapa lapis (`graph/gcn`).

9 Menilai dan Memeras Skor

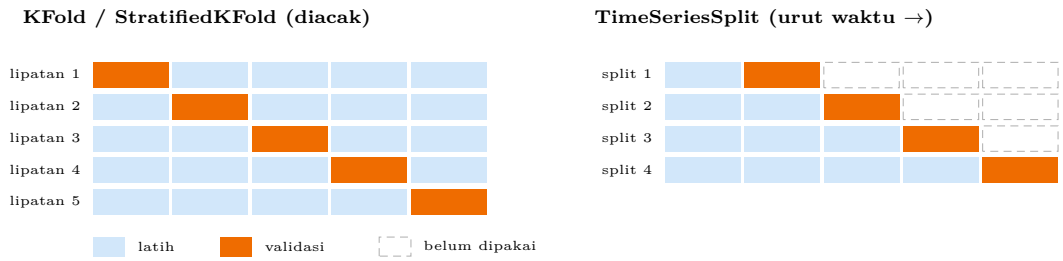
Warna bab ini: latih / kelas 0 validasi / kelas 1 tebakan benar tebakan salah

Skor validasi menentukan semua keputusan. Kalau cara menghitungnya salah, semua keputusan ikut salah.

9.1 Validasi silang (KFold)

Inti. Satu kali split bisa beruntung atau sial. KFold membagi data jadi 5 bagian; model dilatih 5 kali, tiap kali satu bagian berbeda jadi validasi. Skor akhir = rata-rata 5 skor, dan simpangannya menunjukkan seberapa stabil model.

Diagram 14: KFold vs TimeSeriesSplit



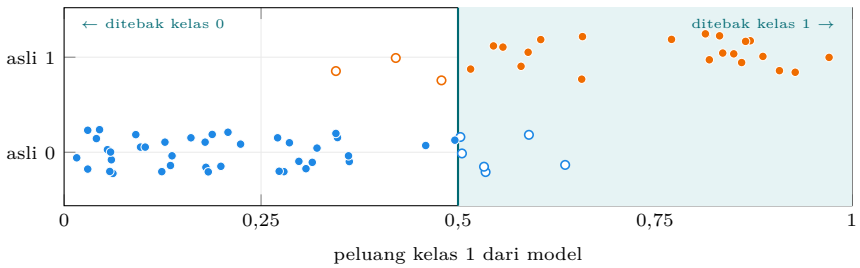
Cara membaca: kiri, setiap bagian kebagian jadi validasi tepat satu kali. Kanan, validasi selalu *sesudah* data latih dan data latih makin panjang; bagian sesudah validasi tidak dipakai supaya model tidak mengintip masa depan.

StratifiedKFold	proporsi kelas di tiap lipatan dijaga sama; wajib bila kelas tidak seimbang (95 % normal, 5 % fraud)
GroupKFold	satu orang/pasien/toko tidak boleh muncul di latih dan validasi sekaligus
TimeSeriesSplit	untuk deret waktu; validasi selalu sesudah latih

9.2 Precision, recall, F1, dan threshold

Inti. Classifier mengeluarkan **peluang**. Kita memilih ambang (*threshold*): di atasnya dianggap kelas 1. Menggeser ambang **menukar** dua jenis kesalahan: alarm palsu (FP) dan yang terlewat (FN).

Diagram 15: Satu set prediksi, threshold 0,5



Titik penuh = tebakan benar; cincin kosong = tebakan salah. **Cara membaca:** cincin biru di daerah kanan = FP (aslinya 0, ditebak 1); cincin jingga di kiri = FN (aslinya 1, ditebak 0). Geser garis ke kiri: FN berkurang, FP bertambah.

$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

threshold	TP	FP	FN	TN	precision	recall	F1
0,3	25	15	0	30	0,62	1,00	**0,77**
0,5	22	6	3	39	0,79	0,88	**0,83**
0,7	14	0	11	45	1,00	0,56	**0,72**

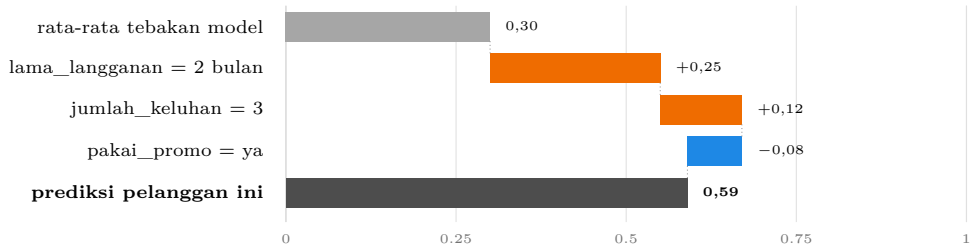
F1 macro	F1 dihitung per kelas lalu dirata-rata; kelas kecil sama penting dengan kelas besar
AUC	kualitas *urutan* peluang (apakah kelas 1 umumnya diberi peluang lebih tinggi); tidak bergantung threshold
MAE	rata-rata $\lvert y - \hat{y} \rvert$; mudah dibaca dalam satuan aslinya
RMSE	$\sqrt{\text{rata-rata } (y - \hat{y})^2}$; kesalahan besar dihukum lebih berat
R^2	1 = sempurna, 0 = setara menebak rata-rata, negatif = lebih buruk dari rata-rata

Jebakan lomba. Pakai metrik yang **persis sama** dengan yang ditulis di soal. Akurasi 95 % bisa tidak berarti apa-apa kalau 95 % data memang kelas 0 (model yang selalu menjawab 0 pun dapat 95 %). Untuk F1, threshold 0,5 belum tentu terbaik; cari di data validasi (`tabular/imbalanced`), jangan di data test.

9.3 Ensembling, tuning, dan SHAP

Ensembling / blending	rata-ratakan peluang beberapa model berbeda jenis (misal LightGBM 0,62, CatBoost 0,48, RandomForest 0,71 → 0,60). Kesalahan mereka tidak sama, jadi saling menutupi	`tabular/ensembling_klasifikasi`, `tabular/blending`
Tuning (Optuna)	mencoba kombinasi kenop dan belajar dari percobaan sebelumnya: area yang skornya bagus dicoba lebih sering. Lebih hemat daripada mencoba semua kombinasi	`tabular/tuning_klasifikasi`

Diagram 16: SHAP: kenapa pelanggan ini ditebak 59 % akan berhenti



Cara membaca: mulai dari rata-rata tebakan model untuk semua pelanggan (0,30). Tiap fitur pelanggan ini mendorong naik (jingga) atau turun (biru); jumlah semua dorongan = prediksi akhir 0,59. Notebook: `tabular/explain_klasifikasi`, `visualisasi/penjelasan_model`.

Kenapa begitu? (ensembling vs tuning, mana dulu) Tuning biasanya menaikkan skor sedikit (sering < 1 poin) dan makan waktu. Ensembling 2–3 model yang sudah lumayan dan berbeda jenis sering memberi kenaikan serupa dengan usaha lebih kecil. Urutan hemat waktu saat lomba: baseline → fitur yang lebih baik → ganti model → ensembling → tuning.

Lembar ringkas satu halaman

Metode	Idenya dalam satu kalimat	Kenop utama
Decision Tree	tanya ya/tidak berulang, potong ruang dengan garis lurus	`max_depth`, `min_samples_leaf`
Random Forest	ratusan pohon dalam dari sampel acak, dirata-rata	`n_estimators`, `max_features`
XGBoost/LightGBM	pohon dangkal berurutan, tiap pohon menebak sisa kesalahan	`learning_rate`, `n_estimators`, `num_leaves`
CatBoost	boosting yang jago kolom kategori	`iterations`, `depth`
LogisticRegression	jumlah berbobot → sigmoid → peluang; batas lurus	`C`
Ridge / Lasso	regresi linear dengan rem bobot	`alpha`
SVC	batas dengan jarak (margin) terbesar; kernel = bisa melengkung	`C`, `kernel`
KNN	suara k tetangga terdekat	`n_neighbors`
MLP / CNN / Transformer	lapisan neuron; pretrained + embedding atau fine-tune	epoch, learning rate, batch size
KMeans	titik ikut pusat terdekat, pusat pindah ke tengah	`n_clusters`
DBSCAN	klaster = daerah padat; sisanya noise	`eps`, `min_samples`
PCA	proyeksi ke arah sebaran terbesar	`n_components`
IsolationForest	sedikit potongan acak untuk terisolasi = anomali	`contamination`
TF-IDF	sering di dokumen ini, jarang di lainnya	`ngram_range`, `min_df`
Exp. smoothing	rata-rata berat ke data terbaru	`smoothing_level` (α)
SARIMAX	AR + selisih + koreksi error + musim + faktor luar	`order`, `seasonal_order`
Fitur lag + LightGBM	deret waktu diubah jadi tabel	lag yang dipakai
SVD (recsys)	orang dan barang jadi vektor selera	`n_factors`, `reg_all`
Louvain	kelompok yang padat di dalam, jarang ke luar	`resolution`

Rumus	
Gini	$1 - \sum_k p_k^2$
Boosting	$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$, h_m menebak $y - F_{m-1}$
Regresi logistik	$z = b_1 x_1 + \cdots + c$; $p = 1/(1+e^{-z})$
TF-IDF	$\text{TF} \times \left(\ln \frac{1+N}{1+\text{df}} + 1\right)$
Cosine similarity	$\vec{a} \cdot \vec{b} / (\|\vec{a}\| \|\vec{b}\|)$
Exponential smoothing	$\ell_t = \alpha y_t + (1-\alpha)\ell_{t-1}$
Matrix factorization	$\hat{r} = \mu + b_u + b_i + \vec{p}_u \cdot \vec{q}_i$
Modularitas	$\sum_c [L_c/m - (d_c/2m)^2]$
Precision / recall / F1	$TP/(TP+FP)$; $TP/(TP+FN)$; $2PR/(P+R)$
RMSE / MAE	$\sqrt{\overline{(y-\hat{y})^2}}$; $\overline{\lvert y-\hat{y}\rvert}$

Kesalahan pemula yang perlu diingat

Gejala	Penyebab	Perbaikan
skor validasi 0,99, skor lomba jauh lebih rendah	kebocoran data (scaler/TF-IDF di-`fit` pada semua data, kolom turunan target)	`Pipeline`; cek kolom satu per satu
KNN/SVC/KMeans hasilnya aneh	fitur tidak diskalakan	`make_pipeline(StandardScaler(), model)`
skor latih jauh di atas skor validasi	overfitting	kurangi `max_depth`/`num_leaves`, naikkan regularisasi
deret waktu skornya terlalu bagus	split diacak / rata-rata bergulir memuat bulan ini	`TimeSeriesSplit`; `shift(1)` sebelum `rolling`
akurasi tinggi tetapi kelas kecil tidak pernah tertebak	kelas tidak seimbang	pakai F1 macro, `class_weight="balanced"`, atur threshold
`learning_rate` diturunkan, skor malah turun	jumlah pohon tidak dinaikkan	naikkan `n_estimators` bersamaan
skor berubah-ubah tiap *Run All*	seed tidak dipasang	`random_state=SEED` di model dan split